Plan van aanpak: communicatievoorkeur per zaak (route C, in OpenKlant)

Doel in één zin: een inwoner ziet zijn zaken en kiest per zaak via welk communicatiemiddel (e-mail, sms, Berichtenbox, brief) hij geïnformeerd wil worden; de keuzes zijn beperkt tot wat de gemeente voor dat soort zaak toestaat, en het OMC kan per zaak opvragen welk kanaal gebruikt moet worden.

Uitgangspunt: we bouwen verder op OpenKlant (route C) als backend en leveren het inwonerscherm als **Next.js-frontend** (wens van de stakeholder) die met die backend communiceert. Alle voorkeuren, kanalen en adressen leven in OpenKlant, dat al `Partij` en `DigitaalAdres` heeft; de frontend bewaart zelf geen gegevens.

Dit plan vertaalt `docs/HvA/OPDRACHT.md` (WORTH × Public Lab) naar werk in deze repo. Het begint bij **het scherm voor de inwoner** en werkt van daaruit naar de data en de API. Spoor 1 (bouwen) staat centraal; spoor 2 (onderzoek & ontwerp) krijgt in §7 de aanknopingspunten.

Leesvolgorde: 1. Het scherm · 2. Hoe het werkt · 3. Wat bestaat al · 4. Ontwerp · 5. Bestanden · 6. Stappenplan · 7. Spoor 2 · 8. Opleveringen · 9. Risico's · Bijlage A: API-specs

---

1. Het scherm waar het om draait

De inwoner (hier: "Anna") ziet twee dingen: hoe ze standaard bericht krijgt per soort zaak, en een lijst van haar zaken met per zaak een keuzelijst.

```
Mijn communicatievoorkeuren                                  Ingelogd als Anna  [TESTLOGIN]
──────────────────────────────────────────────────────────────────────────────────────────
Mijn zaken - hoe wil ik geïnformeerd worden?

Zaak                                Status             Informeer mij via
──────────────────────────────────────────────────────────────────────────────────────────
Kapvergunning Dorpsstraat 3         In behandeling     [ Berichtenbox ▼ ]   ← afwijkend
Kapvergunning Kerklaan 12           Afgerond           [ Standaard (e-mail) ▼ ]
Parkeervergunning 2026-0042         Aangevraagd        [ Standaard (sms) ▼ ]
   keuzes per zaak: alleen kanalen die de gemeente voor dit soort zaak toestaat

Standaard per soort zaak (geldt ook voor nieuwe zaken)
──────────────────────────────────────────────────────────────────────────────────────────
Kapvergunning        [ E-mail ▼ ]    (toegestaan: e-mail, Berichtenbox)
Parkeervergunning    [ Sms ▼ ]       (toegestaan: e-mail, sms)

Mijn contactgegevens                                                  [ + toevoegen ]
──────────────────────────────────────────────────────────────────────────────────────────
E-mail   anna@example.nl          geverifieerd
E-mail   anna.werk@example.nl     niet geverifieerd      [ verifieer ]
Sms      06 12345678              geverifieerd
Brief    Dorpsstraat 3, Amsterdam (uit mijn gegevens)

[ Opslaan ]                       "Voor sms is een geverifieerd telefoonnummer nodig."
```

Waarom **ook** een standaard per soort zaak? Het eerste bericht over een nieuwe zaak ("zaak aangemaakt") gaat de deur uit op het moment dat de zaak ontstaat, vóórdat de inwoner er iets over kon kiezen. Zonder standaard per soort zaak kent het OMC op dat moment geen voorkeur. De voorkeur per zaak is een **uitzondering** daarop.

Toestanden die ontworpen moeten worden: geen zaken, Open Zaak niet bereikbaar, geen toegestaan kanaal voor dit soort zaak, kanaal gekozen zonder geverifieerd adres, opslaan gelukt/mislukt (met `aria-live`-melding).

2. Hoe het werkt, van scherm tot OMC

```
Inwoner (browser)
   |  Next.js-pagina (HTML van de server; het formulier werkt ook zonder JavaScript)
   ▼
Next.js-server  (profiel_service: de serverlaag. Hier, en nergens anders, staan de tokens)
   | ──► Open Zaak  (JWT):    welke zaken heeft deze burger?
   | ──► OpenKlant  (Token):  toegestane kanalen per zaaktype, adressen, voorkeuren lezen en opslaan
   | ──► NotifyNL-verificatie (fake tot Worth levert)
   ▼
OpenKlant: Communicatievoorkeur  (de API controleert de regels uit §4.2)

OMC bij een event over zaak Z van burger B:
   GET /klantinteracties/api/v1/voorkeurskanaal?zaak=Z&zaaktypeIdentificatie=<identificatie>&partijIdentificator__objectId=B ...  (vo
        ──► "berichtenbox" (bron: zaak)  of  "email" (bron: zaaktype-standaard)  of  bron: geen
   ──► NotifyNL verstuurt via dat kanaal
```

Vijf beslissingen volgen hieruit:

1. **De frontend is een Next.js-app met een eigen serverlaag.** Zowel Open Zaak (JWT-secret) als OpenKlant (`Token`, die de héle API opent) hebben geheimen die nooit naar de browser mogen. De browser praat daarom alleen met Next.js; de Next.js-server praat met OpenKlant en Open Zaak (server naar server, dus geen CORS). Zie 4.5.
2. **De zaken komen uit Open Zaak, niet uit OpenKlant.** OpenKlant kent zaken niet; het kent alleen verwijzingen (`Onderwerpobject` met `codeObjecttype="zaak"`). Er komt dus een kleine client naar Open Zaak, in de Next.js-server.
3. **Een “soort zaak” = zaaktype.** De gemeente beheert per zaaktype welke kanalen toegestaan zijn. Het OMC werkt al met zaaktype-identificaties (whitelists), dus heet het model in OpenKlant `Zaaktype` en is `Zaaktype.identificatie` gelijk aan de zaaktype-**identificatie** uit Open Zaak. Daarmee is de eerdere open vraag “hoe koppelt het OMC een event aan de juiste instelling” opgelost. (In `OPDRACHT.md` heet dit “scope”; in code, API en dit plan gebruiken we de duidelijkere naam `Zaaktype`.)
4. **Twee niveaus van voorkeur**: standaard per soort zaak (zaaktype) en uitzondering per zaak. Het OMC lost op in die volgorde: zaak → zaaktype-standaard → geen (het OMC houdt dan zijn eigen volgorde).
5. **De regels zitten in de backend, niet in het scherm.** Het scherm toont alleen wat de API toestaat (een beperkte keuzelijst) en geeft de API-foutmeldingen weer. Zo kan geen enkele client, ook een latere vervanger van dit scherm, een ongeldige voorkeur opslaan, en kan het OMC erop vertrouwen.

3. Wat OpenKlant al heeft

Gecontroleerd in de code (`src/openklant/components/klantinteracties/`):

Nodig voor het scherm	Bestaat al?	Hergebruik
Burger	Ja	`Partij` + `PartijIdentificator` (`bsn`); zoeken via `partijIdentificator__*`
Meerdere e-mailadressen/telefoonnummers	Ja	`DigitaalAdres` (`email`, `telefoonnummer`, `overig`)
Geverifieerd	Deels	`DigitaalAdres.verificatie_datum` + API-filter `isGeverifieerd`; de verificatie-uitvoering via NotifyNL niet
Postadres voor brief	Ja	`Partij.correspondentieadres_*`
Verwijzing naar een zaak	Patroon	`Onderwerpobject` verwijst via `onderwerpobjectidentificator` (`codeObjecttype="zaak"`, `codeSoortObjectId="uuid"`, `objectId`) naar een zaak zonder FK; dit patroon volg je
Zaken van de burger opvragen	**Nee**	Nieuwe client naar Open Zaak, in de Next.js-server (§4.4); OpenKlant zelf hoeft Open Zaak niet aan te roepen
Zaaktypen met toegestane kanalen, voorkeuren	**Nee**	Nieuw: modellen, admin en API (`zaaktypen`, `communicatievoorkeuren`)
Eén globale voorkeur	Deels	`Partij.voorkeurs_digitaal_adres`, `is_standaard_adres`: geen vervanging, wel mogelijke terugval
Beheerscherm gemeente	Deels	Django-admin (inlog, 2FA, OIDC)
Inwonerpagina	**Nee**	Nieuw: Next.js-app in `profiel_service/`. De bestaande admin, Django-templates en webpack zijn er niet voor inwoners
Burger vinden en adressen beheren via de API	Ja	`partijen` (filters `partijIdentificator__*`) en `digitaleadressen` (volledige CRUD; filters `verstrektDoorPartij__uuid`, `isGeverifieerd`; `verificatieDatum` is schrijfbaar)
Toegang tot de API	Ja	`Authorization: Token <token>` (`TokenAuth`). Een token geeft toegang tot de **hele** API, zonder beperking per partij: de Next.js-server moet zelf afdwingen dat een inwoner alleen eigen gegevens ziet

Berichtenbox is geen adres en staat niet als template-kanaal in de OMC-documentatie (daar alleen `EMAIL`, `SMS`, `LETTER`).

4. Ontwerp

4.1 Datamodel



- `Zaaktype` is een **lokale registratie, geen kopie** van het zaaktype in Open Zaak: alleen de `identificatie` (de sleutel die het OMC kent), de `naam` die de inwoner leest en de toegestane kanalen. Het is **data**, zodat de gemeente er zelf een kan toevoegen. In de UI heet het “soort zaak” (of wat het gebruikersonderzoek oplevert); “zaaktype” is jargon voor de inwoner.
- `zaak_uuid` verwijst zonder FK naar de zaak in Open Zaak, naar het patroon van `Onderwerpobject`. Dat voorkomt een harde koppeling; de consequentie is dat je zelf moet omgaan met zaken die verdwijnen (aanname: de voorkeur blijft bestaan tot opschoning; de bewaartermijn bepaalt spoor 2).
- Hiërarchie is bewust weggelaten (opdracht: plat houden).

4.2 Validatieregels (de eigenlijke businesslogica; tests eerst)

1. Het kanaal moet voor het zaaktype `is_toegestaan` zijn.
2. `email` / `sms`: er moet een `DigitaalAdres` van **dezelfde partij** zijn met juiste soort (`email` resp. `telefoonnummer`) en `verificatie_datum` gevuld.
3. `brief`: de partij heeft een ingevuld `correspondentieadres`.
4. `berichtenbox`: geen adres nodig (afspraak Worth).
5. Bij een voorkeur per zaak: de zaak hoort bij deze partij en de zaaktype-identificatie van de zaak = `zaaktype.identificatie`. Dit controleert de Next.js-server (`lib/voorkeuren/service.ts`) bij Open Zaak, niet de OpenKlant-API. De API zelf accepteert `zaak_uuid` dus **ongecontroleerd** (aanname: alleen de vertrouwde Next.js-server schrijft hier met een token; leg dat vast in `API_BESCHRIJVING_OMC.md`). Wil je dat niet, dan moet OpenKlant zelf Open Zaak aanroepen (een Python-client met `zgw_consumers`), wat de API afhankelijk maakt van Open Zaak.
6. Gemeente zet een kanaal uit: bestaande voorkeuren met dat kanaal worden bij het opvragen genegeerd (terugval) en in het scherm gemarkeerd, niet verwijderd (aanname).

Regels 1-4 en 6 horen in de **OpenKlant-API** (serializer-validatie op basis van `models/validators.py`), zodat elke client dezelfde regels krijgt. De API antwoordt bij een overtreding met HTTP 400 en een Nederlandse melding per veld; de frontend toont die melding.

4.3 API

Onder `/klantinteracties/api/v1/` , `Authorization: Token <token>` , volgens het viewset-patroon. De frontend heeft geen toegang tot de database, dus alles wat het scherm nodig heeft moet hier beschikbaar zijn:

Endpoint	Doel	Gebruiker
`zaaktypen` (alleen lezen; met `toegestaneKanalen`; filter `identificatie`)	welke kanalen mag de inwoner kiezen per soort zaak (beheer via de admin, §4.6)	frontend
`communicatievoorkeuren` (CRUD; filters `partij__uuid`, `zaaktype__identificatie`, `zaak`)	voorkeuren lezen/schrijven, met de regels uit §4.2	frontend, tests
`voorkeurskanaal` (alleen lezen)	**antwoord voor het OMC**	OMC
`partijen` (bestaat)	de inwoner vinden op BSN (`partijIdentificator__*`), `correspondentieadres` voor brief	frontend
`digitaleadressen` (bestaat)	contactgegevens tonen en toevoegen; `verificatieDatum` zetten na verificatie	frontend

De verificatie zelf (code versturen en controleren via NotifyNL) is geen OpenKlant-endpoint: de Next.js-server doet dat via `lib/verificatie/` (nep tot Worth documentatie levert) en zet daarna `verificatieDatum` op het adres.

Aanroep door het OMC (kort, één antwoord):

```
GET /klantinteracties/api/v1/voorkeurskanaal
      ?zaak=<zaak-uuid>
      &zaaktypeIdentificatie=<zaaktype-identificatie>
      &partijIdentificator__codeSoortObjectId=bsn
      &partijIdentificator__objectId=<bsn>

200 { "kanaal": "berichtenbox", "bron": "zaak",
      "digitaalAdres": null }                      # bron: "zaak" | "zaaktype" | "geen"
```

Leg vast wat bij `bron: "geen"` gebeurt (aanname: het OMC houdt dan zijn eigen volgorde) en log nooit het BSN.

4.4 Zaken ophalen uit Open Zaak

Nieuwe module `profiel_service/lib/openzaak/` (TypeScript, alleen op de server, `import "server-only"`). Authenticatie is een ZGW-JWT (HS256, ondertekend met het client-id en het secret die je van de begeleider krijgt, bijvoorbeeld met de bibliotheek `jose`); die gegevens staan alleen in `.env.local`. De aanroepen hieronder zijn gecontroleerd tegen de OpenAPI-specs van de testomgeving (Zaken API 1.5.1, Catalogi API 1.3.1; downloadlinks in bijlage A):

- **Zaken van een burger:** `GET /zaken/api/v1/zaken?rol__betrokkeneIdentificatie__natuurlijkPersoon__inpBsn=<bsn>&expand=zaaktype,status.statustype&pageSize=100` met header `Accept-Crs: EPSG:4326` (de header staat in de spec en in `docs/HvA/NOTIFY_TESTOMGEVING_API.rst`). De JWT gaat mee als `Bearer`. Ook bruikbaar: `rol__omschrijvingGeneriek` (bijv. `initiator`, `belanghebbende`) om te bepalen welke zaken “van” de burger zijn (aanname: `initiator` en `belanghebbende`; machtigingen vallen buiten het prototype), en `startdatum` / `einddatum__isnull` om lopende zaken te tonen. Voor veel filters is er ook `POST /zaken/_zoek`.
- **Geen N+1:** met `expand=zaaktype` komt het volledige zaaktype (met `identificatie` en `omschrijving`) mee onder `_expand.zaaktype`, en met `status.statustype` ook de huidige status. Eén call volstaat dus voor het hele scherm. Cache toch per zaaktype-URL, en verifieer de precieze nesting van `_expand` met een echt antwoord.
- **Zaaktypen vullen:** voor het prototype via de fixture of de admin. Een latere import kan uit `GET /catalogi/api/v1/zaaktypen?status=definitief`, dat per zaaktype `identificatie` en `omschrijving` geeft (meerdere versies delen dezelfde `identificatie`, dus dedupliceer daarop). Dat is een admin-actie of script in Python en dus een uitbreiding; `Zaaktype.identificatie` = `identificatie`.
- **Let op:** `Zaak.communicatiekanaal` / `communicatiekanaalNaam` is het kanaal waarlangs de zaak **binnenkwam**, geen voorkeur; niet hergebruiken. `informeren` bestaat in de Catalogi API alleen op het **statustype**, niet op het zaaktype (de OMC-gitbook noemt het soms bij zaaktype): de OMC-regel “mag deze status een bericht sturen” blijft dus los van jullie voorkeuren.
- `fake.ts` met fictieve zaken voor Anna, in dezelfde vorm als het echte antwoord (`url`, `uuid`, `identificatie`, `omschrijving`, `zaaktype`, `startdatum`, `einddatum`, `status`, `_expand`), zodat je zonder testomgeving kunt ontwikkelen en testen. Schakel tussen fake en echt met één omgevingsvariabele (`USE_FAKE_OPENZAAK`).

4.5 Frontend: Next.js met serverlaag

De stakeholder wil een frontend in Next.js die met de backend communiceert. Dat vervangt het eerdere uitgangspunt van een Django-app met templates. De app staat in `profiel_service/` (repo-root) met een eigen `package.json`; de Node-versie volgt `.nvmrc` (24). Gebruik de actuele stabiele Next.js die `create-next-app` aanbiedt (App Router, TypeScript) en lees de documentatie van **die** versie: conventies verschillen per hoofdversie.

- **Server-first.** Pagina's zijn Server Components die hun gegevens op de server ophalen; de browser krijgt HTML. Opslaan gaat via Server Actions in een gewoon `<form action={...}>`, zodat het formulier ook zonder JavaScript werkt (controleer dat met JavaScript uit). Gebruik `"use client"` alleen voor kleine interactieve stukken, zoals een `aria-live`-melding.
- **Geheimen alleen op de server.** `OPENKLANT_API_TOKEN`, de Open Zaak-gegevens en `SESSION_SECRET` staan in `.env.local` (niet in git) en worden alleen gelezen in modules met `import "server-only"`. Geef nooit een variabele het voorvoegsel `NEXT_PUBLIC_`: Next.js neemt die op in de bundel die naar de browser gaat.
- **Identiteit komt uit de sessie, nooit uit het verzoek.** Omdat één OpenKlant-token de hele API opent, bepaalt `requireSession()` welke partij het is. Een pagina of Server Action mag nooit een `partijUuid` uit een formulierveld of URL vertrouwen. Controleer de sessie in elke pagina en actie, niet alleen in middleware.
- **Contract uit de API.** Genereer de TypeScript-types uit `src/openklant/components/klantinteracties/openapi.yaml` (bijvoorbeeld met `openapi-typescript`, als npm-script `types:openklant`) en schrijf de vorm van API-antwoorden niet met de hand over. Na elke API-wijziging: eerst het schema genereren (Django), dan de types (Next.js).
- **Fouten van de backend tonen.** De OpenKlant-API geeft per veld een Nederlandse melding bij HTTP 400; toon die bij het juiste veld. Is Open Zaak niet bereikbaar, toon dan wel de standaarden en contactgegevens, met een duidelijke melding bij de zaken.
- **Mocks zijn herkenbaar.** Mock-login (`MOCK_LOGIN=true`, nooit in productie), fake zaken en nep-verificatie krijgen elk een zichtbare markering in het scherm.
- **Geen CORS.** Alleen de Next.js-server roept OpenKlant en Open Zaak aan; laat CORS in OpenKlant uit.
- **Vormgeving.** Het scherm in §1 is een eerste schets; de uiteindelijke UI volgt het wireframe van gemeente Amsterdam (zie 4.6). Vraag de stakeholder naar een huisstijl of design system. Voor Nederlandse overheidsfrontends is het NL Design System (bijvoorbeeld de Utrecht-componenten) gebruikelijk, met toegankelijkheid ingebouwd; dat is een keuze, geen vereiste.

Bewust niet gekozen: de browser rechtstreeks met OpenKlant of Open Zaak laten praten (het token zou in de browser staan en OpenKlant kent geen inwonerauthenticatie). Een tweede alternatief is dat OpenKlant de zaken zelf ophaalt en aan de frontend geeft; dan praat de frontend met één backend, maar krijgt OpenKlant een Open Zaak-client en endpoints per ingelogde inwoner. Bespreek dat met de stakeholder als die één backend-adres wil.

4.6 UI voor de gemeente

Django-admin: `Zaaktype` met inline `ZaaktypeKanaal`. Dat is een scherm met inlog en voldoet aan "via een scherm of API" (opdracht). Een eigen scherm is een verbetering voor later; het hoofdwerk zit bij de inwoner. De UI wordt gerealiseerd vanuit het verkregen wireframe door gemeente Amsterdam. Hiervoor moet er eerst een gebruikersonderzoek verricht worden om te achterhalen welke wireframe gebruikers het meeste aantrekt.

5. Bestandsarchitectuur

`NIEUW` = aanmaken, `WIJZIG` = bestaand bestand aanpassen. Paden vanaf de repo-root; tussen haakjes het voorbeeld om te kopiëren.

5.1 Frontend: Next.js-app

```
profiel_service/                          NIEUW  Next.js-app (App Router, TypeScript) naast src/, met eigen package.json
├── package.json, package-lock.json, tsconfig.json, next.config.ts, eslint-configuratie   (maakt create-next-app)
│                         scripts: dev, build, start, lint, test, types:openklant
├── .env.example          NIEUW  alleen variabelenamen, nooit waarden: OPENKLANT_BASE_URL, OPENKLANT_API_TOKEN,
│                         USE_FAKE_OPENZAAK, OPENZAAK_BASE_URL, OPENZAAK_CLIENT_ID, OPENZAAK_SECRET,
│                         USE_FAKE_VERIFICATIE, MOCK_LOGIN, SESSION_SECRET
├── .env.local            NOOIT COMMITTEN  de echte waarden
├── app/
│   ├── layout.tsx        <html lang="nl">, skiplink, header, zichtbare TESTLOGIN-banner
│   ├── page.tsx          stuurt door naar /voorkeuren (zonder sessie naar /login)
│   ├── login/page.tsx    mock-login: kies een testpersoon; alleen met MOCK_LOGIN=true
│   ├── voorkeuren/page.tsx  hoofdscherm (Server Component): zaken + standaard per soort zaak
│   ├── voorkeuren/actions.ts  Server Actions ("use server"): voorkeur opslaan en verwijderen
│   ├── contactgegevens/page.tsx, actions.ts   lijst, toevoegen, verifiëren
│   └── error.tsx, not-found.tsx, loading.tsx
├── components/           KanaalKeuze.tsx, ZaakRij.tsx, ZaaktypeStandaard.tsx, TestloginBanner.tsx (+ *.module.css)
├── lib/                  modules met geheimen beginnen met `import "server-only"`
│   ├── config.ts         leest en controleert de omgevingsvariabelen
│   ├── session.ts        mock-sessie in een httpOnly-cookie; requireSession() geeft de partijUuid
│   ├── openklant/client.ts  fetch met `Authorization: Token ...`; een functie per resource uit §4.3
│   ├── openklant/schema.d.ts  GEGENEREERD uit openapi.yaml (npm run types:openklant); niet met de hand bewerken
│   ├── openzaak/client.ts   ZGW-JWT + zaken van de burger (§4.4)
│   ├── openzaak/fake.ts     fictieve zaken voor Anna, zelfde vorm als het echte antwoord
│   ├── verificatie/client.ts, fake.ts   interface + nep-implementatie tot Worth documentatie levert
│   └── voorkeuren/service.ts  zaken + zaaktypen + voorkeuren samenstellen; regel 5 uit §4.2
└── **/*.test.ts(x)       Vitest + Testing Library, naast de code (de twee acceptatiescenario's als Playwright-test is optioneel)
```

Er is geen SCSS/webpack uit de Django-repo nodig: de app heeft een eigen opmaak (CSS Modules of het gekozen design system).

5.2 Backend: OpenKlant

```
src/openklant/components/klantinteracties/
├── models/communicatievoorkeuren.py       NIEUW  Zaaktype, ZaaktypeKanaal, CommunicatievoorkeurR   (zie models/actoren.py: verwante m
├── models/constants.py                    WIJZIG TextChoices Voorkeurskanaal: email|sms|berichtenbox|brief
├── models/validators.py                   WIJZIG regels §4.2
├── models/__init__.py                     WIJZIG `from .communicatievoorkeuren import *  # noqa` (bestaand sterimport-patroon)
├── models/tests/factories/communicatievoorkeuren.py   NIEUW  ZaaktypeFactory, ZaaktypeKanaalFactory, CommunicatievoorkeurFactory   (
├── models/tests/factories/__init__.py     WIJZIG zelfde sterimport als hierboven
├── models/tests/test_communicatievoorkeuren.py        NIEUW  een test per validatieregel uit §4.2   (zie models/tests/test_partijen.
├── migrations/00NN_*.py                   NIEUW  via makemigrations: stap 3 `zaaktype` + `zaaktypekanaal`, stap 4 `communicatievoork
├── admin/communicatievoorkeuren.py        NIEUW  ZaaktypeAdmin + inline ZaaktypeKanaal (kanaalbeheer gemeente); Communicatievoorkeur
├── admin/__init__.py                      WIJZIG `from .communicatievoorkeuren import *  # noqa`
├── api/serializers/communicatievoorkeuren.py          NIEUW  ZaaktypeSerializer (met `toegestane_kanalen`), CommunicatievoorkeurSeri
├── api/filterset/communicatievoorkeuren.py            NIEUW
├── api/viewsets/communicatievoorkeuren.py, voorkeurskanaal.py   NIEUW  ZaaktypeViewSet is alleen lezen   (zie viewsets/digitaal_adre
├── api/urls.py                            WIJZIG router.register (`zaaktypen`, `communicatievoorkeuren`) + pad voorkeurskanaal
├── api/tests/test_communicatievoorkeuren.py, test_voorkeurskanaal.py   NIEUW
├── metrics.py                             WIJZIG counters
└── openapi.yaml                           WIJZIG gegenereerd: bin/generate_schema_for_component.sh klantinteracties

src/openklant/fixtures/portal_demo.json    NIEUW  zaaktypen (kapvergunning, parkeervergunning), toegestane kanalen, testpartij Anna m

docs/HvA/
├── PLAN_VAN_AANPAK.md                     dit bestand
├── ONTWERPKEUZE_ROUTE_C.md                NIEUW  onderbouwing van de keuze voor OpenKlant + onderzoeksbevindingen (oplevering)
├── API_BESCHRIJVING_OMC.md                NIEUW  korte beschrijving hoe OMC bevraagt (oplevering)
├── onderzoek/{gebruikersonderzoek.md, privacy_avg.md, wireframes/}   NIEUW  (spoor 2)
└── DEMO.md                                NIEUW  demoscript
CHANGELOG.rst                              WIJZIG
.github/workflows/portaal.yml              NIEUW  (optioneel) lint, test en build van profiel_service/ op Node 24; eigen bestand zoda
```

Er is bewust **geen** wijziging nodig in `conf/base.py`, `urls.py`, `pyproject.toml`, het `Dockerfile` of `docker/setup_configuration/data.yaml`: de frontend is geen Django-app en OpenKlant roept Open Zaak niet aan.

Gecontroleerd tegen de repo (2026-10-21): alle genoemde bestaande bestanden en mappen bestaan (`models/validators.py`, `admin/__init__.py`, `api/urls.py`, `metrics.py`, `openapi.yaml`, `src/openklant/fixtures/`, `CHANGELOG.rst`, `.nvmrc`); `profiel_service/` bestaat nog niet.

Praktische regels voor deze indeling:

- **Eén bestand per laag, genoemd naar het onderwerp (backend).** De drie nieuwe modellen horen bij elkaar en staan daarom samen in `communicatievoorkeuren.py` (models, factories, admin, serializers, filterset, viewsets, tests). Dat volgt het bestaande patroon: `actoren.py` bevat `Actor`, `Medewerker`, `GeautomatiseerdeActor` en `OrganisatorischeEenheid`. Maak dus niet per model een eigen bestand; splits pas als een bestand onoverzichtelijk wordt.
- **De Next.js-app is een los project.** Eigen `package.json`, `package-lock.json` en `node_modules` in `profiel_service/`. De `package.json` in de repo-root is de webpack-build van de Django-admin en blijft ongemoeid; draai de frontend-opdrachten dus in de app-map (`cd profiel_service && npm run dev`). Controleer vóór je commit met `git check-ignore -v profiel_service/.env.local profiel_service/node_modules profiel_service/.next` dat geheimen en build-output genegeerd worden: `create-next-app` schrijft een eigen `.gitignore`, maar de `.gitignore` in de root kent voor geheimen alleen `/.env`.
- **Poorten en backend lokaal.** Next.js draait op `:3000`; OpenKlant draai je met `python src/manage.py runserver 8001`. De Docker-container op `:8000` is de gepubliceerde upstream-image en kent jullie nieuwe endpoints niet. `OPENKLANT_BASE_URL` wijst dus naar `http://localhost:8001/klantinteracties/api/v1`.
- **Fixtures.** Laad de demofixture met een expliciet pad, zoals `bin/generate_demo_data.sh` ook doet: `python src/manage.py loaddata src/openklant/fixtures/portal_demo.json`.
- **Geen notificaties** voor de nieuwe resources: ze krijgen geen `NotificationViewSetMixin` en dus ook geen wijziging in `kanalen.py` of `src/notificaties.md`. Een notificatie bij een gewijzigde voorkeur, zodat het OMC kan abonneren in plaats van bij elk bericht te vragen, is een latere uitbreiding.

Let op de migratienummering: lokaal is `0048_...` de laatste migratie; de `latest`-image bevat al een `0049_internetaken_actoren_verbose_name`. Geef je migratie een beschrijvende naam en reken op hernummeren bij een upstream-merge.

6. Stappenplan: waar beginnen we

We werken in kleine commits naar `dev` (gitmoji + issue-nummer, zie de skill-referentie `conventions.md`). Na elke stap: aan de backend-kant `python src/manage.py check` en de relevante tests, aan de frontend-kant (in `profiel_service/`) `npm run lint` en `npm test`. De volgorde zorgt dat we **vroeg iets kunnen laten zien**.

Stap 1 - Omgeving draaiend (`docs/HvA/HVA_INSTALL.rst`). `docker compose up`, `.env`, `check`. Let op: `--no-build` draait de gepubliceerde image, niet jouw code; draai je wijzigingen via `python src/manage.py runserver 8001`. Of draai `docker compose up -d db` Als je dan een error krijgt over permissies: `docker compose exec db psql -U postgres -d openklant -c "ALTER DATABASE openklant OWNER TO openklant; GRANT ALL ON SCHEMA public TO openklant;` Voor de frontend heb je Node 24 nodig (`.nvmrc`); controleer met `node -v`.

Stap 2 - Next.js-project en statisch scherm (eerste demo-resultaat). Maak vanuit de repo-root het project met `npx create-next-app@latest profiel_service` (TypeScript, App Router, ESLint; zonder `src/`-map, want de repo heeft al een `src/`). Haal de voorbeeldpagina weg en bouw `app/voorkeuren/page.tsx` met hardcoded nepdata (de drie zaken uit §1), opmaak met CSS Modules en alleen de zichtbare “TESTLOGIN”-banner in `app/layout.tsx` (`lang="nl"`). Controleer: `npm run dev` en open `http://localhost:3000/voorkeuren`, daarna `npm run lint` en `npm run build`, en `git check-ignore -v profiel_service/.env.local` (zie §5.2). Deze stap raakt Django niet en heeft geen database, geen OpenKlant en geen Open Zaak nodig. Toets het scherm meteen met spoor 2.

Stap 3 - Gemeente-kant in OpenKlant: `Zaaktype` + `ZaaktypeKanaal`, admin en `zaaktypen`-API. Maak `models/communicatievoorkeuren.py` met deze twee modellen, de migratie, de factories en de admin met inline. Een beheerder zet in de admin voor *kapvergunning* e-mail en Berichtenbox open. Voeg de alleen-lezen `zaaktypen`-API toe (serializer met `toegestane_kanalen`, filterset, viewset, router, test) en genereer `openapi.yaml` opnieuw. Maak hier ook de demofixture `src/openklant/fixtures/portal_demo.json` (zaaktypen, toegestane kanalen, testpartij Anna met adressen) en laad hem met `python src/manage.py loaddata src/openklant/fixtures/portal_demo.json`. Controleer met `curl` en een token (uit `.env`, niet in een bestand in git) dat `GET /zaaktypen` de kanalen teruggeeft.

Stap 4 - `Communicatievoorkeur` + validatie + API. Voeg het model toe aan hetzelfde `models/communicatievoorkeuren.py`, maak de migratie en de validators uit §4.2 (regels 1-4 en 6) met een test per regel in `models/tests/test_communicatievoorkeuren.py` (niet-toegestaan kanaal, ongeverifieerd adres, adres van andere partij). Admin met `raw_id_fields` (anders faalt `openklant.E001`). Voeg dan de `communicatievoorkeuren`-API toe: serializer die de validators aanroept (HTTP 400 met een Nederlandse melding per veld), filterset, viewset, router en API-tests. Genereer `openapi.yaml` opnieuw.

Stap 5 - Frontend aan de backend koppelen, nog met fake zaken. Maak `lib/config.ts`, `lib/openklant/client.ts` (server-only, `Authorization: Token`) en genereer de types met `npm run types:openklant`. Voeg de mock-login toe (`app/login`, `lib/session.ts`: kies een testpersoon, alleen met `MOCK_LOGIN=true`) en `requireSession()`. `lib/openzaak/fake.ts` levert Anna's zaken; `lib/voorkeuren/service.ts` combineert ze met zaaktypen en voorkeuren (inclusief regel 5 uit §4.2); de Server Action slaat de keuze op en toont API-fouten bij het veld. Nu is het scherm **end-to-end bruikbaar** zonder Open Zaak. Controleer met JavaScript uit dat het formulier nog werkt, en dat in de netwerktab van de browser geen verzoek naar OpenKlant of Open Zaak gaat.

Stap 6 - `voorkeurskanaal`-API voor het OMC. Schrijf eerst twee tests: (a) de opdracht: Anna heeft voor kapvergunning "e-mail" gekozen → antwoord `email`, bron `zaaktype`; (b) uitzondering: voor één zaak "berichtenbox" → antwoord `berichtenbox`, bron `zaak`. Daarna viewset, filterset, router, `openapi.yaml`, `API_BESCHRIJVING_OMC.md`.

Stap 7 - Contactgegevens en verificatie. Pagina `app/contactgegevens` voor adressen toevoegen (`POST /digitaleadressen`) en verifiëren; `lib/verificatie/fake.ts` met een nep-code die daarna `verificatieDatum` zet via de OpenKlant-API. Markeer dit in het scherm als nep. Echte NotifyNL pas als Worth documentatie en testgegevens levert.

Stap 8 - Echte Open Zaak. Vervang fake door `lib/openzaak/client.ts` tegen de testomgeving (`USE_FAKE_OPENZAAK=false`); neem geen tokens op in code of commit (zie `docs/HvA/NOTIFY_TESTOMGEVING_API.rst` en de regels in de skill). Foutafhandeling voor "Open Zaak niet bereikbaar" in het scherm.

Stap 9 - Afronden en demo. Toegankelijkheid, foutmeldingen in het Nederlands, `ruff`, `npm run lint` / `npm test` / `npm run build`, volledige backend-testsuite, `CHANGELOG.rst`, demoscript, ontwerpkeuze-document, PR naar `dev`.

Bij tijdnood is stap 2-6 het minimale prototype: gemeente-kanaalbeheer, scherm voor de inwoner en het OMC-antwoord. Verificatie (7) en echte Open Zaak (8) mogen nep blijven, mits je dat in de demo zegt.

7. Spoor 2: onderzoek & ontwerp

Spoor-2 onderdeel	Wat het oplevert voor de bouw	Waar
Gebruikersonderzoek (medewerkers + inwoners)	Welke soorten zaak, welke kanalen, welke woorden; willen inwoners per zaak én standaard kiezen?	`docs/HvA/onderzoek/gebruikersonderzoek.md`
Wireframes, getoetst	Het scherm in §1 is een eerste schets: toets en verbeter vóór stap 5	`docs/HvA/onderzoek/wireframes/`
Privacy & AVG	Grondslag, bewaartermijn van voorkeuren/adressen, verantwoordelijke (gemeente of centrale dienst); beïnvloedt verwijderen en logging	`docs/HvA/onderzoek/privacy_avg.md`
Begrijpelijke taal	“Zaaktype” begrijpt een inwoner niet; test “soort zaak”, “waarover”, enz.; ook voor de uitleg bij sms/brief	labels in `Zaaktype.naam`, React-componenten, helpteksten

Geef spoor 2 vroeg het statische scherm (stap 2) en later de admin met demodata, zodat tests over echte schermen gaan.

8. Opleveringen en definition of done

Oplevering (uit de opdracht)	Waar
Werkend prototype kanaalbeheer + burgervoorkeuren	admin (gemeente), Next.js-scherm `/voorkeuren` (inwoner, lokaal `http://localhost:3000`), API
Onderbouwde ontwerpkeuze (verder bouwen op OpenKlant) + onderzoeksbevindingen	`ONTWERPKEUZE_ROUTE_C.md`
Korte API-beschrijving voor OMC	`API_BESCHRIJVING_OMC.md` + gegenereerd `openapi.yaml`
Wireframes + samenvatting gebruikersonderzoek	`onderzoek/`
Korte demo voor Worth (en gemeente)	`DEMO.md` + demofixture

Technisch:

- [ ] De twee scenario’s uit stap 6 zijn automatische tests en slagen.
- [ ] Elke validatieregel uit §4.2 heeft een test.
- [ ] Het scherm toont alleen de door de gemeente toegestane kanalen per soort zaak.
- [ ] Het scherm werkt zonder JavaScript, met toetsenbord, en met duidelijke Nederlandse foutmeldingen.
- [ ] Backend: `manage.py check` schoon (incl. `openklant.E001`), `ruff check src` en `ruff format src` choon, `python src/manage.py test src` groen.
- [ ] Frontend (in `profiel_service/`): `npm run lint`, `npm test` en `npm run build` slagen.
- [ ] `openapi.yaml` opnieuw gegenereerd en `npm run types:openklant` opnieuw gedraaid; OMC-endpoint eschreven.
- [ ] Geen tokens, echte BSN’s of echte persoonsgegevens in code, fixtures, tests of commits. Geen enkele ariabele met `NEXT_PUBLIC_` bevat een geheim, en in de netwerktab van de browser gaan alleen verzoeken naar e Next.js-server.
- [ ] Mock-login is zichtbaar gemarkeerd en alleen actief met `MOCK_LOGIN=true` (nooit in productie).

9. Risico’s en keuzes die onderbouwd moeten worden

Risico / keuze	Toelichting
Het OMC leest nu geen voorkeuren	Het OMC gebruikt een vaste volgorde (e-mail, sms, brief). Jullie leveren het antwoord-endpoint en de beschrijving, niet de OMC-aanpassing (.NET, andere repo). Zeg dat in de demo; simuleer eventueel met `curl`.
Eerste bericht vóór de keuze	Daarom bestaat de standaard per soort zaak (§1). Zonder die standaard blijft het eerste bericht op de OMC-volgorde draaien.
Verder bouwen op OpenKlant (route C)	Voordeel: hergebruik van `Partij` / `DigitaalAdres` /token-auth, aansluiting op wat al bij gemeentes draait. Nadeel: OpenKlant is een registratiecomponent zonder inwoner-authenticatie of inwonerpagina’s; het inwonerscherm is daarom een apart Next.js-portaal dat met een servertoken met OpenKlant praat.
Afhankelijkheid van Open Zaak	Zaken staan daar; zonder testtoegang werk je met de fake-client. Plan het vragen van toegang vroeg.
`Zaaktype.identificatie` is uniek binnen een catalogus	De identificatie is uniek binnen een catalogus, niet noodzakelijk over catalogi heen. Aanname: één catalogus in de testomgeving; bij meerdere catalogi moet `Zaaktype` ook de catalogus kennen.
Open Zaak-koppeling	Alleen gelezen (zaken, zaaktypen); we schrijven niets terug. Filters en `expand` zijn gecontroleerd in de spec, maar nog niet tegen een echt antwoord.
Nieuwe API = contractwijziging	`KLANTINTERACTIES_API_VERSION` in `conf/api.py`, changelog; overleg wat past bij een fork.
Persoonsgegevens	Alleen testdata; log UUID’s, geen BSN, adressen of namen.
Upstream-merge	De hele frontend staat in `profiel_service/` en raakt `src/` niet. In OpenKlant komen alleen nieuwe bestanden (`communicatievoorkeuren.py` per laag) en kleine aanpassingen in bestaande registratiebestanden (`__init__.py`, `api/urls.py`, `constants.py`, `validators.py`, `metrics.py`).
Eén token = de hele API	`TokenAuth` kent geen rechten per partij. Een fout in de Next.js-server (bijvoorbeeld een `partijUuid` uit een formulierveld vertrouwen) toont of wijzigt gegevens van een andere inwoner. Leid de partij altijd af uit de sessie en schrijf een test die dit afdwingt.
Twee toolchains en een contract	Python/Django én Node/Next.js, met `openapi.yaml` als enige koppeling. Als we vergeten na een API-wijziging het schema of de types te genereren, dan breekt het scherm pas bij gebruik. Zet daarom beide stappen in de checklist (§8) en overweeg een CI-controle die de gegenereerde bestanden vergelijkt.
Next.js-conventies veranderen per versie	Request-API’s, caching en de naam en plek van authenticatie-hooks verschillen tussen hoofdversies. Pin de versie in `package-lock.json` en volg de documentatie van die versie, niet een tutorial.
Productie valt buiten het prototype	De app moet in productie ergens draaien (Node-server of container) met eigen secrets, HTTPS en echte inwonerauthenticatie in plaats van de mock-login. Zeg dat in de demo.

Bijlage A: API-specs van de testomgeving

Openbaar op te halen (geen token nodig voor de spec zelf); de aangeleverde `.../schema/`-pagina's zijn ReDoc-weergaven van deze bestanden. Gebruik `.json` of `.yaml` in plaats van de HTML-pagina wanneer je een spec wilt downloaden of vergelijken.

API	Versie in spec	Spec
Catalogi (Open Zaak)	1.3.1	`https://openzaak.test.notifynl.nl/catalogi/api/v1/schema/openapi.json` (of `.yaml`)
Zaken (Open Zaak)	1.5.1	`https://openzaak.test.notifynl.nl/zaken/api/v1/schema/openapi.json`
Klantinteracties (OpenKlant)	0.7.0	`https://openklantv2.test.notifynl.nl/klantinteracties/api/v1/openapi.json` (of `.yaml`)
Contactgegevens (OpenKlant)	1.1.1	`https://openklantv2.test.notifynl.nl/contactgegevens/api/v1/openapi.json`
Open Notificaties	1.0.0	`https://opennotificaties.test.notifynl.nl/api/v1/schema/openapi.json` (of `.yaml`)

Wat voor dit plan relevant is:

- **Zaken:** `GET /zaken` met `rol__betrokkeneIdentificatie__natuurlijkPersoon__inpBsn`, `rol__omschrijvingGeneriek`, `expand`, `zaaktype`, `startdatum` / `einddatum`-filters, `page` / `pageSize` (max 500), header `Accept-Crs`. Verder `POST /zaken/_zoek`, `/rollen`, `/statussen`.
- **Catalogi:** `GET /zaaktypen` (`identificatie`, `omschrijving`, `status=definitief`), `GET /statustypen` (met `informeren`, `isEindstatus`, `volgnummer`).
- **Klantinteracties (test):** zelfde 29 paden als de fork, `digitaleadressen` met `isGeverifieerd`, `verificatieDatum` en `verstrektDoorPartij__partijIdentificator__*`; `partijen` met `partijIdentificator__*` en `expand`. Daarmee kan het OMC een partij/adres op BSN vinden zonder extra werk.
- **Open Notificaties:** `abonnement` (`callbackUrl`, `auth`, `kanalen: [{naam, filters}]`, `sendCloudevents`, `cloudeventFilters`), `kanaal` (`naam`, `documentatieLink`, `filters`), `POST /notificaties`, `POST /cloudevents`. Security-schema `JWT-Claims`. Handig om te weten hoe het OMC zich op `zaken` / `objecten` / `besluiten` abonneert (zie `integrations.md`); jullie hoeven hier niets aan te bouwen.

Download specs in een eigen map (bijv. de scratchpad of `/tmp/<naam>/`), niet in de repo, en lees ze met een script dat je buiten die map bewaart.